EECS3311
Course Project Stage 2 Instructions

AI Human Collaborative Implementation

Implementation period: October 20-November 10, 2026

Submission due date: November 10, 2026

1. Objective

In Stage 2, you will implement the AI agent-based system designed in Stage 1. Your goal is to produce a working application whose features, classes, interactions, and design patterns can be traced to your Stage 1 design.

The Project Instructions and Stage 1 Instructions remain the baseline for the project's functionality and design requirements. The additional Stage 2 requirements below address implementation, GitHub activity, reporting, feature demonstration videos, and API-key protection.

Using AI development tools is mandatory. You remain responsible for understanding, reviewing, integrating, and validating all submitted code, including AI-generated code.

2. Required Functionality and Design

Your implemented system must satisfy the same minimum requirements established in Stage 1:

- **GUI:** Provide a graphical user interface through which users can access the application's major functionality.
- **CLI:** Provide a command-line interface through which users can access the agent's major functionality.
- **At least 10 meaningful features:** Implement at least 10 distinct, non-trivial features. Login, Logout, Exit, About, Change Password, and similarly simple operations do not count toward this minimum.
- **At least 5 distinct design patterns:** Meaningfully implement at least five design patterns covered in the course. Repeated uses of the same pattern do not count as additional distinct patterns.
- **LLM integration:** Integrate at least one LLM model (**you can use any cheap or free models**) into the working system.
- **Meaningful agent behavior:** The AI component must participate in behavior such as reasoning, planning, tool use, retrieval, memory, decision making, or multi-step task execution. An application that only forwards a prompt to an LLM API and displays the response is insufficient.

Implement the features proposed in your Stage 1 report, subject to the documented changes permitted in Section 3. The minimum of 10 features is not permission to silently omit additional features proposed in Stage 1. Account for every originally proposed feature in your report.

Features must work in the submitted application. Interface mockups, empty methods, hard-coded demonstration responses, and unconnected classes do not constitute completed functionality. Design patterns must be evident in the code and solve identifiable design problems; naming a class after a pattern is insufficient.

3. Changes from Stage 1

You may add new features, replace existing features, and use different design patterns when implementation reveals a better approach. These changes must preserve the minimum requirements above and be explicitly explained in the Stage 2 report.

For every added, replaced, removed, or substantially modified feature, document:

1. The original Stage 1 feature and its ID, or indicate that the feature is new.
2. The implemented feature and its behavior.
3. What changed and why the change was necessary or beneficial.
4. How a replacement addresses the original user need, or how the project scope has changed.
5. The affected use cases, classes, methods, and design patterns.

For every new, replaced, or substantially modified design pattern, document:

6. The original pattern, if applicable.
7. The pattern actually implemented.
8. The participating classes and their roles.
9. The design problem addressed.
10. Why the replacement or addition is appropriate and how it affects the architecture.

Changes are allowed, but unexplained differences between the Stage 1 design and the implementation are not acceptable. New features do not automatically excuse missing original features, and adding a feature does not reduce the required number of design patterns.

4. GitHub Source Code and Weekly Commits

Continue using the **same public GitHub repository** created for Stage 1. Submit the complete source code directly to this repository; a source-code ZIP file or report alone does not satisfy the code submission requirement.

Weekly code activity

Your project repository must contain **at least five meaningful code commits in each of the following reporting periods**:

Period	Dates, inclusive	Minimum code commits
1	October 20-26, 2026	5
2	October 27-November 2, 2026	5
3	November 3-9, 2026	5

Dates are interpreted in Toronto local time. The requirement applies to the project repository. Each period must satisfy its own minimum; extra commits in one period do not compensate for missing commits in another.

A meaningful code commit changes implementation or test code and represents actual development progress, such as implementing functionality, fixing a defect, refactoring, or adding relevant tests. Empty commits, report-only edits, formatting-only changes, and artificially splitting a trivial change do not satisfy this requirement. Use descriptive commit messages and push your commits to GitHub during the relevant period. Do not postpone all repository uploads until the deadline.

Penalty: If any reporting period fails to meet the minimum of five meaningful code commits, 5 points will be deducted from the Stage 2 grade in total. This is a single 5-point deduction, not 5 points per missing commit or per deficient period.

Include a weekly commit summary in your report, with links to the commits used to satisfy each period.

5. AI Human Collaborative Development

Document the major AI tools used during implementation. For each tool, identify:

- Tool or platform name.
- LLM/model name.
- Features, classes, or development tasks for which it was used.
- Total Token cost and money cost

Stage 2 Implementation Report

Submit a **Stage 2 Implementation Report in PDF/word format (named "stage2-report")** in the GitHub repository. It must describe the actual submitted implementation and contain the following sections.

1 Project overview and implementation status

Briefly describe the application, its users, its AI/LLM integration, and the current working functionality. Identify limitations and incomplete functionality honestly.

2 Feature to implementation mapping

Account for every Stage 1 feature and every new feature. Identify the classes and files that implement each feature, their responsibilities, relevant methods, design patterns, and demonstration video.

Mapping field	Example entry
Feature ID and name	F01 - [Feature name]
Stage 1 feature/reference	[Original feature ID or New]
Status or change	[Implemented/Modified/Replaced/Not implemented]
Implemented classes/files and responsibilities	[Classes, files, responsibilities]
Key methods	[Methods]
Design patterns	[Patterns or N/A]
Video link/timestamp	[Link]

For each implemented feature, briefly explain how the classes collaborate from user input to the final result. State whether the feature is deterministic, AI-based, or hybrid. Do not list class names without explaining their role.

3 Implemented design patterns

Explain at least five distinct implemented design patterns. For each pattern, provide:

- Pattern name and design problem addressed.
- Participating classes/interfaces and their pattern roles.
- Relevant source files and methods.
- How the implementation follows the pattern and why the pattern is useful.
- Whether it matches Stage 1; if it differs, explain the change and rationale.

Include an updated UML class diagram reflecting the implemented system. Update affected use-case and sequence diagrams when significant changes make the Stage 1 diagrams inaccurate.

4 Changes and rationale

Provide a consolidated record of feature, architecture, class, and pattern changes using the structure **Original design -> Implemented change -> Reason -> Impact**. Include all new features and all feature or pattern replacements described in Section 3.

5 AI tools and collaboration log

Include the tool/model information and representative interactions required in Section 5.

6 Development and verification summary

Include the weekly commit summary, checks performed to verify the implementation, important defects found and corrected, and remaining limitations. Stage 2 requires a working implementation and basic verification; the full deterministic testing and KUMA agent-validation requirements remain part of Stage 3 under the Project Instructions.

7 Demonstration index

Provide a feature-by-feature index linking each implemented feature to its demonstration video or clearly labeled video segment.

Feature Demonstration Videos

Every implemented feature, including each new or replacement feature, must have a video demonstration.

You may submit a separate video for each feature or combine demonstrations into longer recordings, provided every feature has a clearly labeled segment and a direct link with a timestamp in the report.

Each demonstration must:

- Identify the feature ID and name.
- Show how the user invokes it and the required input.
- Show the feature running in the actual application and its resulting output.
- Explain briefly what the feature accomplishes and where AI is involved, if applicable.
- Demonstrate relevant error or alternative behavior where appropriate.

Collectively, the recordings must show both the GUI and CLI working. Slides, screenshots, or descriptions alone are not substitutes for a running demonstration. Ensure all video links are accessible to the instructor and teaching assistants without an access request. Do not expose API keys, credentials, or other secrets in recordings.

Local Configuration and API Key Protection

Do not commit or push LLM API keys or other credentials to GitHub. Store actual keys in a local configuration file, such as `.env` or `config.local.json`, and add that file to `.gitignore` before committing project files.

For example, if your application uses `.env`:

```
# Local secrets
.env
.env.*
!.env.example
config.local.json
```

Commit a safe example configuration containing placeholders only, such as:

```
# .env.example - contains no real credentials
LLM_API_KEY=YOUR_LOCAL_API_KEY_HERE
```

Your README must explain how to copy the example configuration, enter a key locally, and run the application. Never hard-code real keys in source code or include them in reports, collaboration logs, screenshots, videos, or committed output files.

Adding a file to `.gitignore` does not remove it from existing commits. If a key is accidentally pushed, revoke or rotate it immediately and remove the exposed credential from the repository and its history. Deleting the current file alone does not make the exposed key safe.

Submission Checklist

By November 10, 2026, commit and push the following to the same public course-project GitHub repository:

11. **Complete source code**, including the GUI, CLI, AI integration, and supporting code needed to run the system.
12. **README**, including prerequisites, dependency installation, local configuration, GUI/CLI launch commands, and example usage.
13. **Dependency/build files** and safe sample data needed to reproduce the application.
14. **Stage 2 Implementation Report (PDF)** with feature/class/pattern mappings, change explanations, AI collaboration records, and weekly commit summary.
15. **Feature demonstration videos or accessible video links**, indexed by feature in the report.
16. **`.gitignore` and a safe example configuration**, with no real API keys or credentials committed.

Submit the GitHub repository URL through the course submission system. Verify that the repository is public, all required files have been pushed, the README instructions work, and all demonstration links are accessible.

Your submission should allow a reviewer to trace **Stage 1 requirement -> implemented feature -> participating classes and methods -> design patterns -> running demonstration**.